CIBIL Check Agent – Project Guide

Location: `UdemyTask/cibil_agent/` | Stack: Streamlit, LangChain, Ollama (gemma4), Oracle (python-oracledb)

The project had no logging, so logging was added. Logs now go to the terminal and to `logs/cibil_agent.log`. A `Dockerfile` and `.dockerignore` were also added for packaging. The app has not yet been run end to end with Oracle and Ollama together.

> VS Code may still mark the `langchain` and `oracledb` imports as unresolved. The code runs fine; the editor just hasn't picked up the `.venv` change. Run **Developer: Reload Window** to clear it.

1. Checking the logs

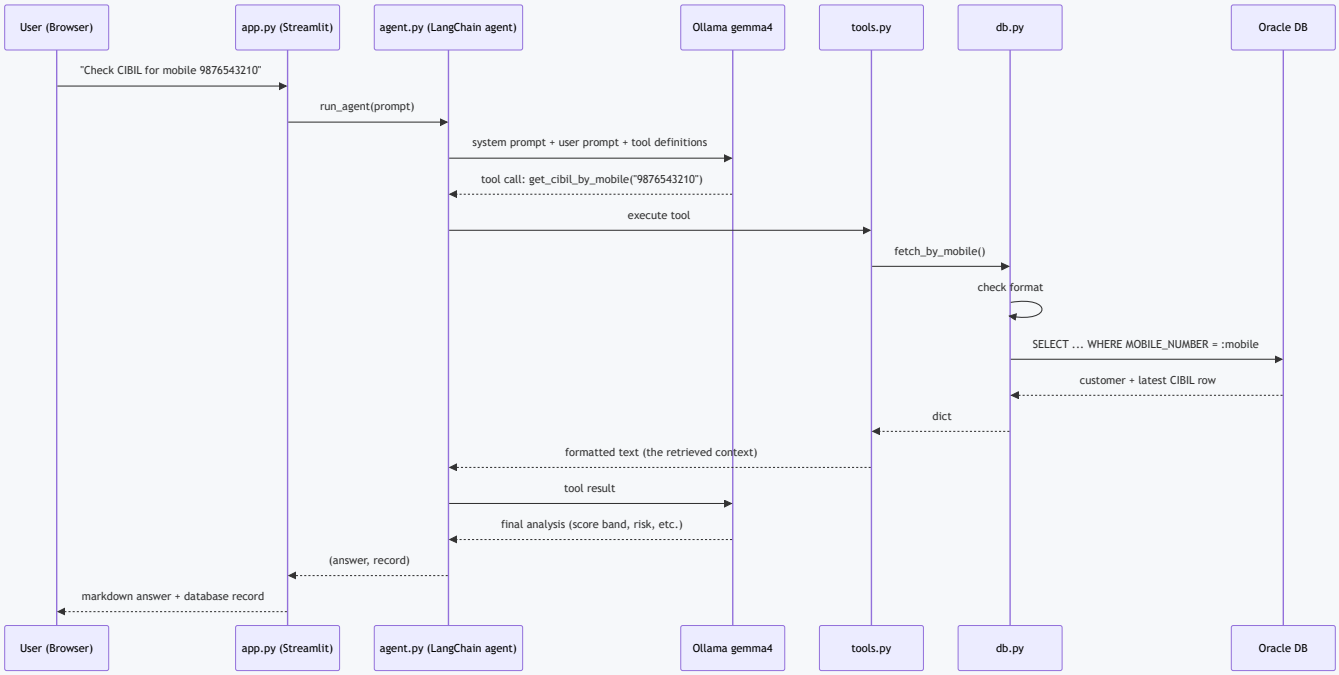
Where	How
Terminal running `streamlit run app.py`	Log lines appear live
`UdemyTask/cibil_agent/logs/cibil_agent.log`	Keeps up to 5 old files of 5 MB each
Follow the file live	`Get-Content logs\cibil_agent.log -Wait -Tail 50`
More detail	Set `LOG_LEVEL=DEBUG` in `.env`
Ollama's own logs	`%LOCALAPPDATA%\Ollama\server.log`

What gets logged:

- **Agent:** each prompt received (length only, not the text), the model used, total time, and whether a record was found. It also logs when the fallback is used and full errors when the agent fails.
- **Tools:** which tool the model called, the last 4 characters of the customer ID or mobile, and any input that was rejected.
- **Database:** pool creation, each query's condition, whether a row was found, how long it took, and connection failures.

Full mobile numbers, the password and CIBIL details are never logged.

2. How it works



1. **User (browser)** types: "*Check CIBIL for mobile 9876543210*" into the Streamlit chat.
2. **app.py** calls `run_agent(prompt)`.
3. **agent.py** sends the system prompt, user prompt and tool definitions to **Ollama gemma4**.
4. **gemma4** replies with a tool call: `get_cibil_by_mobile("9876543210")`.
5. **tools.py** runs the tool, which calls `db.fetch_by_mobile()`.
6. **db.py** checks the format, then runs `SELECT ... WHERE MOBILE_NUMBER = :mobile` on **Oracle**.
7. Oracle returns the customer plus their latest CIBIL row. db.py returns it as a dict, and tools.py turns it into text (the retrieved context).
8. The agent sends the tool result back to gemma4, which writes the final analysis (score band, risk, etc.).
9. **app.py** shows the markdown answer and the database record to the user.

What each file does

File	Role
`config.py`	Loads `.env` and sets up logging.
`db.py`	Oracle connection pool and lookups.
`tools.py`	The two lookup tools the model can call.
`agent.py`	The LangChain agent: model, tools and system prompt.
`app.py`	Streamlit chat page and sidebar.

- **`db.py`:**
  - Joins `VAG_CUSTOMER_MASTER` to the customer's latest `VAG_CIBIL_REPORTS` row.
  - Uses bind variables, so input can't be injected into the SQL.
  - Checks the customer ID and mobile format before querying.
  - Cuts the bureau summary text to 2,000 characters.
- **`tools.py`:**
  - The docstrings on the two tools tell the model when to use each one.
  - The last record found is kept so the UI can show it.
- **`agent.py`:**
  - The model chooses which tool to call.
  - If `gemma4` turns out not to support tool calling, the code pulls the ID or mobile out of the prompt itself, queries Oracle, and asks the model only to write the summary.
- **`app.py`:** chat history, a spinner while it works, the database record under each answer, and sidebar buttons to test the database and clear the chat.

The "retrieval" part is fetching live rows from Oracle at question time and giving them to the model as context. There is no vector database; that isn't needed for exact ID or mobile lookups. The system prompt tells the model to answer only from that data.

3. Running locally

```
cd C:\varathaa\AI_WS\PY_WS\UdemyTask\cibil_agent
..\..\.venv\Scripts\Activate.ps1

# 1. Ollama is running and the model is present
ollama list

# 2. Database connection and a sample lookup
python db.py C1001
python db.py 9876543210

# 3. Start the UI (opens http://localhost:8501)
streamlit run app.py
```

Try these in the chat:

- `Check CIBIL for customer C1001`
- `CIBIL for 9876543210`
- `hello` : it should ask for an ID or mobile.
- `C1' OR '1'='1` : it should be rejected.

To let others on your network use it, run `streamlit run app.py --server.address 0.0.0.0` and share `http://<your-ip>:8501`.

4. Packaging and deploying as a web app

Deploying online needs three things:

1. **Oracle must be reachable from the server.** `orabetst*-scan.sg.gbs.pro` is an internal host, so public clouds (Streamlit Cloud, Render, a public Azure or AWS app) can't reach it. In practice this means deploying on your company's internal servers or cloud network.
2. **Ollama must be reachable from the server.** Either run Ollama on that server or a GPU VM and set `OLLAMA_BASE_URL`, or use a cloud model such as `gpt-oss:120b-cloud`.
3. **Secrets go in as environment variables**, never inside the image.

Option A: Docker (recommended)

```
cd UdemyTask\cibil_agent
docker build -t cibil-agent:1.0 .

# Run, passing secrets from .env and pointing at the host's Ollama
docker run -d --name cibil-agent -p 8501:8501 `
  --env-file .env `
  -e OLLAMA_BASE_URL=http://host.docker.internal:11434 `
  -v ${PWD}\logs:/app/logs `
  cibil-agent:1.0

docker logs -f cibil-agent   # live logs
```

To put it on a server:

- Push the image to your registry (`docker tag` then `docker push`).
- Run it on an internal server, Kubernetes, or a cloud service like Azure Container Apps or AWS ECS inside your company network.
- Keep the secrets in the platform's secret store rather than a `.env` file.

Option B: Linux VM without Docker

```
python3 -m venv venv && source venv/bin/activate
pip install -r requirements.txt
# create .env on the server (chmod 600 .env)
nohup streamlit run app.py --server.port 8501 --server.address 0.0.0.0 --server.headless true &
```

For anything long-lived, run it as a `systemd` service rather than with `nohup` .

Before exposing it to users

- **Put Nginx or IIS in front** with HTTPS and WebSocket upgrade enabled (`proxy_set_header Upgrade $http_upgrade; proxy_set_header Connection "upgrade";`). Streamlit needs WebSockets to work.
- **Add authentication.** This app shows credit data, so use SSO or the reverse proxy's login, or Streamlit's built-in `st.login` with OIDC.
- **Use a read-only Oracle account** with `SELECT` on only these two tables.
- **Change the password** that was pasted into the chat, and keep `.env` out of git and out of images (both ignore files already exclude it).
- **Expect slow answers on CPU.** `gemma4` is a 9.6 GB model; a GPU host helps, and you may want to raise `max` in the connection pool for more users.